

order by with default clause

```
select
    employee_id,
    first_name,
    salary,
    sum(salary) over(order by employee_id
        range between unbounded preceding and current row -- default query that
gets added because of order by
    ) as c_salary
from employees.employees;
# 691400.00
# 691400.00
```

Range vs rows in window function

```
select
    employee_id,
    first_name,
    salary,
    sum(salary)
    over(
        order by first_name
        range between unbounded preceding and current row -- default query
    ) as c_salary
from employees.employees
where first_name = 'James'
order by first_name asc;
```

```
select
    employee_id,
    first_name,
    salary,
    sum(salary)
    over(
        order by first_name
        rows between unbounded preceding and current row -- default query
    ) as c_salary
from employees.employees
where first_name = 'James'
```

order by first_name **asc**;

first option (rows)

127 -> 2400

131 -> 4900

second option (range)

127 -> 4900

131 -> 4900

get me the next highest salary for every employee

and then the difference between them

select *, next_higher_sal - salary **from** (

select

employee_id,

first_name,

hire_date,

salary,

lead(salary, 1) **over**(**order by** salary) **as** next_higher_sal,

lag(salary, 1) **over**(**order by** salary) **as** prev_lower_sal

from employees.employees

) t;

with next_higher_salary_cte **as** (

select

employee_id,

first_name,

hire_date,

salary,

lead(salary, 1) **over**(**order by** salary) **as** next_higher_sal, -- no duplicate

value

lag(salary, 1) **over**(**order by** salary) **as** prev_lower_sal -- no duplicate

value

from employees.employees

)

select *, next_higher_sal - salary **from** next_higher_salary_cte;

select

employee_id,

```
    first_name,  
    hire_date,  
    salary  
from employees.employees  
order by salary;
```

```
# Create index f_name_idx in table employees  
# PK is an index already  
# FK as index  
# list down all indexes for my employees table  
show index from employees.employees;  
-- employee id, job id, dept id, manager id (already present)
```

```
# to see what is happening internally for MySQL, we use explain keyword  
# it shows the steps performed for fetching the result in MySQL  
explain  
select * from employees.employees  
where last_name = 'James';
```

```
# create an index on top of your first_name  
create index first_name_idx on employees.employees (first_name);  
select * from employees.employees;
```